

AEGIS-Q: A UMA-Aware Secure Edge File-Encryption Runtime

Changjong Kim

changjong5238@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Yongseok Son

sysganda@cau.ac.kr

Chung-Ang University
Seoul, Republic of Korea

Ehan Sohn

ehanjsohn@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Sunggon Kim

sunggonkim@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Abstract

Unified-memory edge devices co-locate local databases, encrypted logs, cache-manifest updates, and encrypted state on shared DRAM/storage. A secure file-encryption runtime must place cryptographic work, not merely accelerate it: GPU work can steal bandwidth from foreground storage and durable publication. Its thesis is that secure edge storage should expose one durable authenticated format with a CPU-first AES-GCM data plane and an elastic GPU/PQC maintenance lane. AEGIS-Q explores that boundary as a boundary-aware UMA FUSE runtime with optional batched PQC key-plane maintenance, executor-local managed-memory use, and fail-closed external-anchor replay-after-advance checks. The core result is asymmetric placement under edge pressure. AES-GCM file records remain CPU-first; ML-KEM/cuPQC work uses the GPU only when it is independent, batched, and slack-tolerant; storage-visible QoS throttles elastic writes rather than weakening publication order. On a Jetson AGX Thor Developer Kit, unthrottled secure-storage pressure reports 9.62 ms SQLite p99 and 6.98 MB/s background writes; AEGIS-Q reports 8.15 ms p99, zero median misses, and 3.02 MB/s background writes, while app-only is 7.25 ms. The same mounted path exercises remount/tamper checks, generation faults, rekey fallback, selected app-crash timing, and telemetry-to-FUSE throttling. This is a scoped edge file-encryption runtime result, not deployed-filesystem or peak-throughput superiority.

Keywords

Post-Quantum Cryptography, Edge Computing, Secure Storage, Unified Memory, TPM, FUSE

1 Introduction

Edge devices increasingly run local databases, encrypted logs, cache-manifest updates, and encrypted storage on shared DRAM/storage. Confidentiality and integrity remain mandatory, but a daemon that treats the integrated GPU as a free accelerator can contend with the foreground storage operations it protects. The systems problem is therefore not filesystem conformance, application scheduling, or throughput encryption in isolation; it is a secure file-encryption runtime that makes CPU/GPU placement, storage-visible

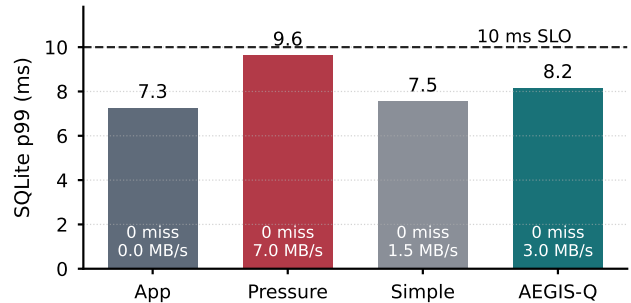


Figure 1: SQLite pressure result for RQ3; labels show deadline misses and background throughput.

Table 1: Capability comparison that defines AEGIS-Q’s design gap. “Policy” means storage-visible accelerator/QoS/replay policy, not implementation maturity.

System	File/blk enc.	Replay chk.	UMA policy	Boundary
Plaintext	✗	✗	✗	Lowerfs baseline only
gocryptfs	✓	✗	✗	User-space encryption baseline
fscrypt	✓	✗	✗	In-kernel file encryption
dm-crypt	✓	✗	✗	Block-device encryption
fs-verity/dm-integrity	△	✗	✗	Read-only file or block integrity
TPM/TEE-backed designs	△	✓	✗	Hardware trust boundary
GPU-storage systems	△	✗	△	Accel. path focus
AEGIS-Q	✓	△	✓	FUSE runtime with scoped evidence

QoS, authenticated publication, and fail-closed replay checks one mounted decision surface. Its thesis is that secure edge storage should expose one durable authenticated format while separating a CPU-first AES-GCM data plane from an elastic GPU/PQC maintenance lane.

Post-quantum key establishment is relevant to long-lived edge data, but it does not replace symmetric storage encryption. ML-KEM establishes shared secrets, whereas AES-GCM remains the appropriate mechanism for bulk data records [8]. SHA-256 can bind committed-prefix anchor material to persisted metadata [7]. The systems challenge is therefore to compose ordinary storage invariants—a key hierarchy that survives remount, nonce/generation state

that remains safe across crashes, ciphertext durability before metadata publication, and fail-closed recovery—with accelerator placement and shared-resource control. Table 1 positions that gap against plaintext, gocryptfs, fscrypt, dm-crypt, fs-verity/dm-integrity, TPM/TEE-backed designs, and GPU-storage systems [4, 5, 11, 15]; the missing cell is storage-visible policy at the edge encryption boundary.

Figure 1 is the paper’s edge-runtime result. Unthrottled secure-storage pressure reports 9.62 ms SQLite p99 and 6.98 MB/s background writes; AEGIS-Q reports 8.15 ms p99 and 3.02 MB/s by throttling only elastic writes. The point is visible storage-layer policy with accelerator-independent format correctness, not peak filesystem throughput. The low frozen-contract throughput row is a known cost boundary, not a hidden general-purpose claim.

AEGIS-Q explores this gap as a UMA-aware secure file-encryption runtime. It adds executor-local managed-memory use and an external-anchor path that can fail closed on replayed state. The claim ties placement to format invariants, QoS policy, and recovery behavior rather than to a GPU microbenchmark or a general-purpose encrypted-filesystem replacement.

The central tension is that the accelerator is both a compute resource and an interference source. GPU-everything file encryption loses on small authenticated writes; KEM-per-block “full PQC” confuses the key plane with the data plane; static placement ignores UMA contention; kernel QoS controls lack file-encryption semantics. AEGIS-Q therefore follows one design insight: accelerator placement must be subordinate to storage correctness and foreground slack. AES-GCM records bind file identifier, block, generation, and length; metadata follows data/journal/checkpoint order; optional GPU work produces format-compatible results or falls back to CPU. The accelerator is an execution choice, not a security boundary.

This paper makes four contributions.

- (1) **C1: Placement-safe storage format.** AEGIS-Q implements authenticated envelopes, generation-bound AEAD, durable mapping publication, and CPU/GPU rules that do not alter the persistent record format.
- (2) **C2: CPU data lane, GPU/PQC maintenance lane.** Measurements explain why AES-GCM writes stay on the CPU, while independent ML-KEM envelope-refresh batches can enter the GPU lane only with slack and telemetry permission.
- (3) **C3: Recovery and replay boundary.** AEGIS-Q separates replayable file witnesses from externally anchored replay-after-advance checks and labels clean-remount, tamper, stale-replay, selected app-crash, and non-claimed power-loss outcomes.
- (4) **C4: Storage-visible QoS control.** AEGIS-Q includes deterministic admission, platform telemetry to mounted-FUSE throttling, and the bounded mounted SQLite result in Figure 1.

The claim boundary excludes unproven `0_DIRECT` NVMe-to-UVM DMA, `io_uring`/eBPF completion bypass, GPU constant-time execution, persistent PCR-bound freshness, power-loss certification, and portability beyond the tested stack.

The rest of the paper follows the structure of a systems design argument. Section 2 states the storage and UMA assumptions, Section 3 presents the format and placement design, Section 4 records implementation choices that determine the boundary, Section 5 states the security and recovery model, Section 6 evaluates the scoped claims, and Sections 7–9 discuss implications, related work, and limits.

2 Background and Scope

2.1 Secure storage primitives

Secure-storage systems choose a protection boundary as well as a cipher. Kernel filesystem encryption, user-space encrypted filesystems, block encryption, and TEEs occupy different boundaries and trust assumptions [4, 5, 11, 15]. Those systems make encryption deployable at mature kernel, user, block, or hardware boundaries; AEGIS-Q asks a different runtime question at the mounted edge boundary. It protects files represented by its own backing format and depends on the kernel, FUSE daemon, OpenSSL/liboqs, and the mount credential. It is therefore evaluated as a placement/QoS/recovery runtime artifact, not as a replacement for fscrypt or dm-crypt.

The cryptographic roles are deliberately distinct. AES-GCM encrypts and authenticates each data block; SHA-256 constructs the committed-prefix root used by the anchor helper; and ML-KEM-768 is a key-establishment primitive rather than a bulk-data primitive [7, 8]. We use ML-KEM only for the optional batched key-plane experiment. The production FUSE data path derives a random file data-encryption key (DEK), wraps it under a mount-derived key, and uses that DEK for AES-GCM. This avoids the atypical and expensive design of performing a KEM operation for every block.

2.2 Why a mounted runtime?

AEGIS-Q distinguishes itself by making placement and recovery policy visible at the same boundary as file encryption. It is not merely gocryptfs/fscrypt plus CUDA/TPM scripts: the research question is whether one mounted runtime can keep authenticated-block publication, storage-visible QoS/admission, optional batched GPU key-plane work, and external replay-after-advance checks under one evidence contract. In that setting, the storage layer exposes policy decisions that conventional encryption layers usually hide. A FUSE daemon can decide whether a maintenance job is latency-sensitive, batch-shaped, or replay-critical; it can also fail closed before exposing a file whose envelope, checkpoint, or external anchor is inconsistent.

The capability matrix on the first page and Figure 2 are not performance rankings. They explain why the paper evaluates AEGIS-Q as a placement/QoS runtime rather than as

a replacement for mature deployed filesystems: existing encryption layers protect bytes, GPU-storage systems optimize data movement, and kernel controls shape I/O priority, but none of those boundaries simultaneously sees file-encryption state, accelerator slack, UMA pressure, and replay-aware recovery exposure.

2.3 Unified memory is not a DMA contract

On a unified-memory SoC, CPU and GPU share DRAM capacity and bandwidth; Jetson-class systems are a representative deployment setting [10]. This makes placement a QoS problem: a GPU job can improve an elastic batch while delaying the foreground path that shares the same memory system. CUDA managed memory makes an allocation accessible to CPU and GPU code, and prefetch is a performance hint; neither API is a permission bit nor a storage-DMA registration protocol. The present implementation allocates managed buffers only inside the CUDA executor, prefetches them around a GPU operation, and synchronizes the stream before CPU reuse. The backing files are accessed with ordinary `pread/pwrite`. It does *not* call `cudaHostRegister`, issue `0_DIRECT` into managed memory, or rely on `cudaStreamAttachMemAsync` as mutual exclusion.

This scope matters because systems work must separate an API’s correctness semantics from an optimization hypothesis. A future direct-I/O path would need an explicit driver/version matrix, pinning and IOMMU evidence, a fault-injection campaign under memory pressure, and a protocol that excludes DMA, CPU, and GPU concurrent access. None of those conditions is established here. The absence of that path is a design constraint, not an unreported optimization.

Auxiliary diagnostics are read under the same rule: same-buffer CUDA checksums show executor visibility after a storage read, not safe registration of every file-cache page, NVMe DMA into managed memory, or deployed page-fault/coherence counter support. AEGIS-Q treats managed memory as executor-local and storage I/O as POSIX sidecar I/O unless a future driver-level campaign proves more.

2.4 Why work placement is asymmetric

GPU acceleration is useful only when it amortizes launch, placement, and synchronization costs. Prior out-of-core systems such as FlashNeuron and Fastensor focus on moving large GPU workloads through storage hierarchies [1, 17]; their complementary staging perspectives are relevant to the batch boundary considered here [1, 17]. GPU-oriented work such as fscrypt-GPU and FlashNeuron motivates separating batch placement from the storage format [1, 18]. They do not validate this FUSE format or its durability protocol. Likewise, storage-oriented systems such as SnuQS motivate explicit staging but do not make an NVMe/UVM correctness claim for AEGIS-Q [12]. The evaluation therefore asks when a secure-storage runtime should use the CPU fast path, GPU elastic lane, or conservative fallback on the tested platform.

The asymmetric answer is useful for system design. Bulk data encryption sits on small writes, SQLite commits, and foreground reads, so launch, staging, synchronization, and UMA contention costs matter. Key refresh, batch encapsulation, and committed-prefix hash maintenance can instead form independent, deferrable jobs. AEGIS-Q’s scheduler asks whether a job is large, independent, and slack-tolerant enough to enter the elastic maintenance lane; otherwise, the CPU path is the correct secure-storage choice.

2.5 Design requirements

The resulting system requirements are:

- (1) **One storage format.** CPU and GPU execution must produce the same authenticated record format, so fallback does not change recovery semantics.
- (2) **Publication before exposure.** A write is not visible merely because ciphertext bytes exist; a validated journal mapping and checkpoint determine exposure.
- (3) **No optimistic accelerator semantics.** CUDA managed memory and prefetch are local executor mechanisms until storage-DMA behavior is separately proven.
- (4) **Explicit replay boundary.** A file-backed witness is an integrity check, not rollback protection; external anchors must be evaluated separately.
- (5) **Admission before acceleration.** A job with no explicit slack or stale telemetry must take the CPU path even if the GPU is present.

These requirements shape the rest of the design. They also explain the paper’s evaluation style: negative controls and scoped failures are part of the evidence because they prevent the runtime from inheriting guarantees from mechanisms it does not actually implement.

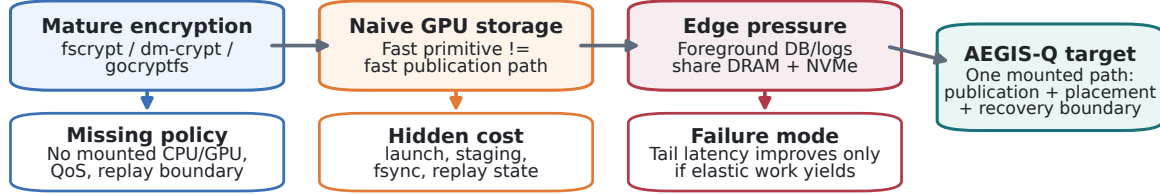
3 AEGIS-Q Design

3.1 Planes and trusted state

Figure 3 separates the runtime into the three mechanisms needed by the edge thesis. First, the persistent data plane uses an AES-256-GCM block format that is identical regardless of executor. Second, the elastic lane admits only independent ML-KEM envelope-refresh and hash-maintenance work after slack and telemetry checks; it does not encrypt ordinary file blocks. Third, the publication/replay plane decides when encrypted state becomes visible and when replayed state must fail closed. Managed memory appears only inside the CUDA executor as a locality mechanism, so placement never changes the storage format or recovery rule.

The design is organized around seven invariants from the threat model and edge-runtime thesis: authenticated key exposure, nonce/generation safety, durable publication ordering, checkpoint exposure, external replay checks, unsupported-mode handling, and controller fallback. These invariants are not the paper’s main novelty by themselves; they are the constraints that allow the placement and QoS

Why a storage-runtime boundary is needed



Design implication: accelerator placement is useful only after the storage format, publication order, and fallback rule are fixed.

Figure 2: Motivating boundary for AEGIS-Q. The open systems problem is not encryption alone; it is the combination of authenticated publication, accelerator placement, foreground QoS, and replay-aware recovery in one mounted path.

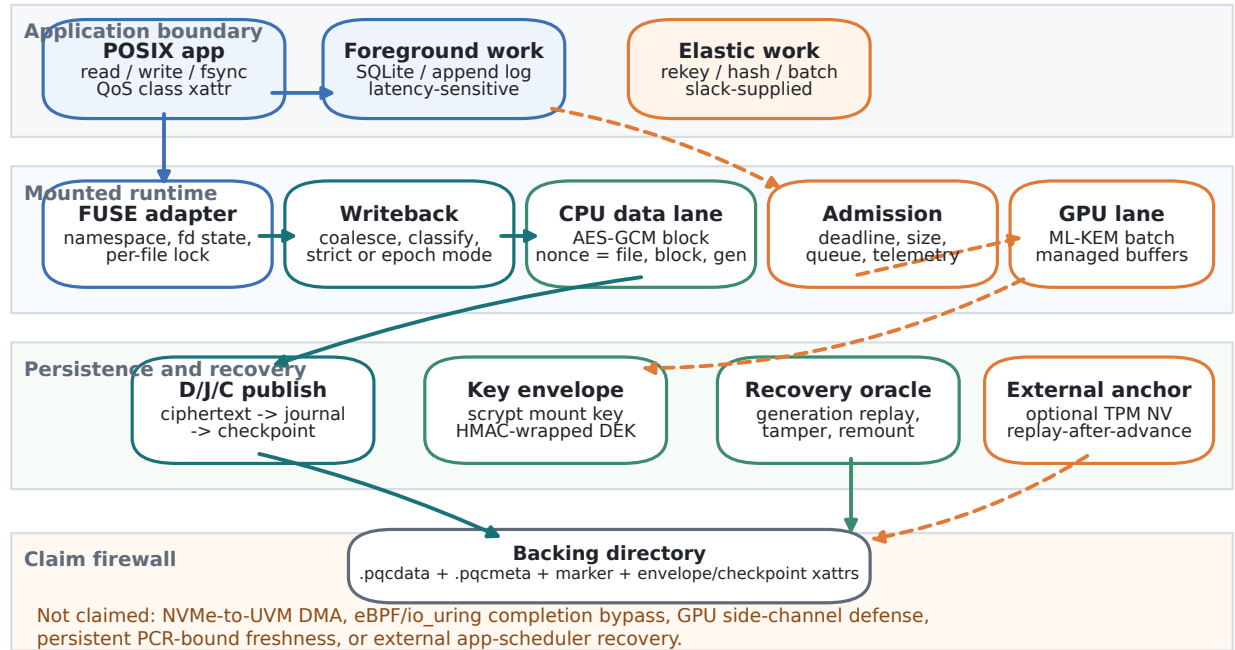


Figure 3: AEGIS-Q runtime architecture after separating the paper’s three boundaries. Solid arrows are implemented mounted paths for foreground reads/writes, publication, recovery, and remount validation; dashed orange arrows are elastic maintenance paths that must pass admission and can fall back to CPU execution. The bottom strip is the claim firewall enforced throughout the paper.

controller to use CPU, GPU, or fallback without changing what has been safely published. Table 2 states the boundary; its POSIX row is only a mounted-workload guardrail, not the research axis.

New roots use OpenSSL scrypt in versioned `.pqc_kdf` metadata (random 32-byte per-filesystem salt; $N = 32768$, $r = 8$, $p = 1$, 64 MiB max memory); roots lacking it use the legacy PBKDF2-HMAC-SHA256 compatibility path.

PBKDF2 is not the new-root default. New files receive random 256-bit DEKs and file identifiers; each DEK is masked under mount-key/file-ID material and HMAC-authenticated before open accepts it. The file identifier, generation, logical block number, and plaintext length are AES-GCM associated data, preventing valid records from moving to different logical positions.

Table 2: Formal storage-protocol invariants, implementation loci, exercised paths, and claim boundaries.

Invariant	Implementation locus	Exercised path	Claim boundary
Envelope authentication before DEK use	<code>metadata.load()</code> verifies the HMAC-protected envelope before <code>pgc.open()</code> installs file state	Clean-remount regression and <code>user.pgc.metadata</code> tamper rejection returning <code>EKEYREJECTED</code>	Remountable authenticated envelope; no hardware-bound credential release
AEAD nonce uniqueness	<code>derive_block_nonce()</code> , <code>build_block_aad()</code> , and <code>do_flush_wbuf_locked()</code> bind file identifier, block, generation, and length	Generation fault matrix, self-test replay regression, clean-remount validation, and daemon cutpoint matrix	Generation-bound record format; physical power loss and broad workload certification remain open
Data-before-mapping publication	<code>do_flush_wbuf_locked()</code> writes <code>.pgcdata</code> , calls <code>fdatasync</code> , appends <code>.pgcmeta</code> , then syncs the journal	Journal self-test, daemon cutpoint matrix, and SQLite selected-boundary campaign	D/J ordering evidence; no physical power-loss or drive-cache certification
Checkpoint exposure rule	<code>logical_size.store()</code> , <code>checkpoint.store()</code> , <code>checkpoint.load()</code> , and <code>remount.ctx.set()</code> expose size/checkpoint state only after journal publication	Checkpoint self-test, clean remount, and stale-replay fail-closed artifacts	Authenticated checkpoint exposure; lower-filesystem xattr/directory ordering is delegated
External-anchor replay condition	<code>checkpoint.store()</code> propagates hardware-anchor failures and <code>checkpoint.load()</code> rejects mismatched anchor state	TPM NV replay-after-advance run fails closed; file-anchor replay is retained as a negative control	Replay-after-advance check only; no persistent PCR-bound lifecycle claim
Unsupported-mode boundary	<code>pgc.open()</code> defaults to FUSE <code>direct.io</code> ; <code>new-target</code> , <code>closed/open-target</code> overwrite, <code>open-source</code> , <code>empty/no-open</code> directory-tree rename, <code>directory fsync</code> , <code>no-open</code> regular-file hard links, relative symlinks, and hidden internal xattrs are bounded callbacks	POSIX scope audit plus current remount/SQLite selected-boundary tests	Ordinary read/write/ <code>fsync</code> , disjoint, same-block full-overwrite, and one partial-overlap writer subset; no shared <code>mmap</code> , <code>open-subtree</code> retargeting, full hard-link lifecycle, or full directory-entry crash durability
Controller safety fallback	<code>pgc.block.job.choose_target()</code> and <code>pgc.admit()</code> default to CPU on missing slack, stale telemetry, size, deadline, or queue-pressure constraints	Scheduler self-test, admission sweeps, and same-run CUPTI/tegrastats-to-mounted-FUSE throttle traces	Implemented conservative fallback; no non-storage foreground scheduler claim

The password-derived mount key is never hardware-released; `scrypt` does not remove weak-password or offline-guessing risk. There is no hardware-backed credential release path for the mount key. The rekey is open-file DEK refresh rather than deployed credential rotation: HMAC envelope rewrap for open files; runtime key epoch is in-memory book-keeping, not a persistent anti-rollback journal. There is no transactional rewrap recovery claim, and password-derived envelope alone is not rollback resistance; stale old envelopes fail only when generation/checkpoint/external-anchor evidence rejects them.

3.2 Durable encrypted-block publication

Writes are coalesced only when they are contiguous. For each affected 4 KiB logical block, AEGIS-Q reads the current authenticated block if necessary, applies the modification, assigns a strictly increasing generation, and derives the GCM nonce from $(fileID, block, generation)$. The commit order is:

- (1) write ciphertext records to `.pgcdata` and call `fdatasync`;
- (2) append generation-to-offset mappings and tags to `.pgcmeta`, then call `fdatasync`;
- (3) update the logical-size xattr; authenticate and write the checkpoint.

Figure 4 makes the publication protocol explicit. It is intentionally not a proof diagram: the upper path names the order that makes data reachable, and the lower path names the failure interpretations that the evaluation is allowed to claim.

The journal is the publication point: a ciphertext tail with no durable mapping is unreachable after recovery. Conversely, a mapping is written only after its ciphertext range has crossed the data durability barrier. `fsync` flushes the

coalescing buffer, waits for local pending work, and calls `fdatasync` on both sidecars before returning. The lower-filesystem matrix assumes `fdatasync` D-before-J ordering and checkpoint xattr visibility after mapping publication; directory-entry operations are rejected rather than certified.

Strict and epoch publication share one accepted-state invariant. Strict publishes data then journal/checkpoint per sync and replays the newest authenticated mapping. Epoch publishes only a complete grouped `syncfs` prefix and discards incomplete tails; it amortizes aligned writers but can lose to wait time, sparse groups, or replay work. Thus epoch changes the barrier schedule, not the nonce, AAD, or replay semantics.

The generation transition is per-file. A publish-ticket serializes reservation turns; before encryption, near wraparound fails with `EFBIG` if the affected block count would pass `UINT64_MAX`. Otherwise AEGIS-Q reserves $[next + 1, next + n]$ in checkpoint/high-water state, may skip interrupted reservations, advances committed generation only after publication, recovers only mappings reachable through the validated boundary, and rejects duplicate epoch records for the same file, block, and generation. Misuse-resistant SIV-style AEAD would trade this state obligation for extra passes; AEGIS-Q instead tests older-generation replay while not claiming physical power-loss, arbitrary application-level concurrency, or broad crash-consistency certification.

Unsupported POSIX modes are part of the design boundary, not the research axis. The scope audit covers the ordinary read/write/`fsync`, scoped rename, directory `fsync`, no-open hard-link, symlink, and bounded writer subsets needed by the mounted workloads; it rejects or scopes out encrypted-file `mmap/msync`, `open-subtree` retargeting,

Publication protocol: data before mapping, mapping before exposure

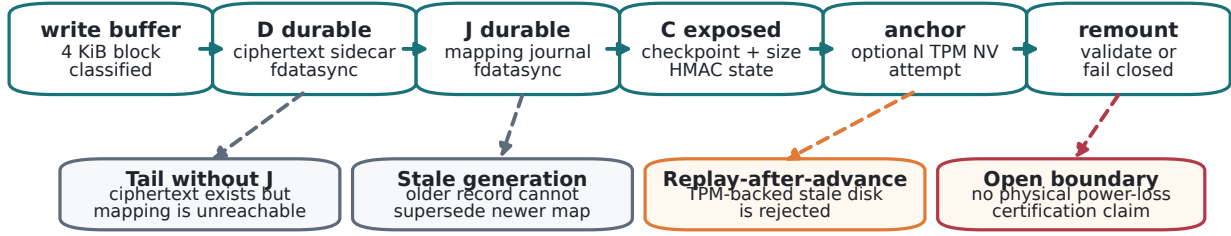


Figure 4: D/J/C/xattr publication protocol. D is the durable ciphertext sidecar state, J is the journal mapping state, and C is the authenticated checkpoint/logical-size exposure state. The bottom row is the claim boundary: tails without J are unreachable, stale generations cannot supersede newer mappings, TPM replay-after-advance fails closed, and physical power-loss certification remains outside scope.

Table 3: Memory-path hardware claims and their exercised status.

Mechanism	Status	Claim retained
cudaMallocManaged + prefetch	Exercised	Managed allocation and explicit device/host prefetch complete.
posix.memalign + cudaHostRegister	Prototype-only	Host-memory pinning and executor-local access checks; not a storage-DMA path.
pread/pwrite sidecars	Exercised	Backing ciphertext and journal use ordinary POSIX I/O.
eBPF tracepoint notification	Not claimed	No eBPF notification path is part of the mounted implementation.
QoS telemetry	SQLite recovery only	Workload/CUPTI traces can drive mounted-FUSE throttling; no application scheduler or non-storage p99 recovery claim.

arbitrary byte-range transactions, directory-entry crash durability, and full crash-time POSIX visibility.

3.3 Managed-memory boundary and scheduler

Table 3 summarizes the claims that the source code actually makes: accelerator placement must be subordinate to storage correctness, CPU/GPU ownership is protected by the FUSE per-file lock and stream completion, and CUDA prefetch is used only to influence locality. No table entry is a statement about NVMe page pinning, cache coherence of a third-party DMA engine, or CUDA attach-mode permissions.

The placement function balances work between the ARM CPUs and the Jetson GPU by classifying each job as foreground-sensitive data-plane work, elastic key-plane work, or unsupported optimistic offload. The current revision does not expose an `O_DIRECT` storage path into GPU memory. The CUDA executor instead uses managed allocation, explicit prefetch, and stream synchronization as locality and ordering controls.

QoS follows Figure 1: unthrottled secure-storage pressure reports 9.62 ms SQLite p99 and 6.98 MB/s background writes; AEGIS-Q reports 8.15 ms p99 and 3.02 MB/s by throttling elastic writes, not by claiming application scheduling or completion bypass.

The producer-facing interface is conservative. A job enters the elastic lane only when its size and batch shape make GPU execution plausible, the producer supplies nonzero relative slack, and telemetry is fresh enough to avoid adding pressure to the foreground path. Missing, zero, or stale slack falls back to CPU with the deferral reason. The mounted path also exposes `user.pqc-qos-class`: latency files are traced but never daemon-slept, elastic files are throttle-eligible, and SQLite sidecars inherit the base-file class.

The evaluated key-plane workflow applies this rule by batching open-file descriptors for ML-KEM envelope refresh, encapsulating on the selected executor, installing 32-byte refreshed DEK material, and rewriting the HMAC-authenticated envelope. It is not a persistent KEM hierarchy or hardware-backed credential lifecycle.

3.4 Replay boundary

The file anchor is an authenticated on-disk record. Because an attacker able to restore the backing directory can restore that record as well, it is explicitly *not* rollback protection. The crash/replay harness treats this behavior as a negative control. The optional hardware backend refuses to mount when its TPM NV index was not provisioned by an administrator; it never creates persistent TPM state implicitly. The hardware claim is narrower: replay after the anchor advances fails closed. A stronger rollback-resistance claim requires a separately protected TPM policy, defined NV authorization, a recovery protocol, and power-loss results for that exact configuration.

Table 4: Implementation boundaries that are easy to misread without an explicit summary.

Mechanism	Current implementation	Boundary
CUDA storage path	Managed allocation with explicit prefetch and stream synchronization	Not an NVMe-to-UVM storage-DMA path
CUDA failure policy	CUDA executor falls back to CPU on failure	Does not change the on-disk record format
Anchor backend	File witness plus optional hardware NV index backend	Hardware path requires administrator pre-provisioning
Scheduler fallback	No-slack jobs route to CPU	No optimistic GPU admission without producer-supplied slack

4 Implementation

AEGIS-Q is implemented in C/CUDA on FUSE3. The source tree splits the FUSE adapter, fd-context lifecycle, writeback/publication, epoch log, recovery, anchor, keyring/KDF, QoS, POSIX policy, and CUDA executors across `code/`; `pqc_fuse.c` is mount-facing. The build enables `-Wall -Wextra -Werror` for C/C++ and builds CUDA targets only when cuPQC is present.

Table 4 summarizes choices that are easy to misread. Publication is a strict data/journal/checkpoint path plus opt-in epoch-redo-log grouping, not an accelerator path. GPU AES-GCM is format-compatible and falls back to CPU, but evaluated data writes are CPU-first. The GPU ML-KEM executor is not an in-place filesystem data path; integrity parity tests do not imply a persisted per-file Merkle tree or side-channel protection.

The key-envelope path is central to remount correctness. The current path generates a random DEK, wraps it under the mount key, HMAC-authenticates the envelope, and rejects failed authentication before ciphertext is read. ML-KEM is optional batched maintenance: the rekey worker can refresh open-file DEK material and persist new HMAC envelopes, but the format does not pretend that a KEM benchmark is a persistent per-file key hierarchy.

The older `pqc_file_key.*` epoch/grace helper remains source-only and is not linked into `pqc_fuse`; evaluated epochs are open-file refresh bookkeeping, not stale-handle, rollback, or TPM/PCR-bound rollback proof.

The implementation also fails closed in two places. A scheduler job with no producer-supplied slack is routed to CPU. A requested TPM backend with no pre-provisioned NV index causes initialization to fail rather than creating a persistent index with undocumented ownership. These are conservative operational choices; neither substitutes for a

complete external foreground-workload telemetry integration or TPM policy design.

4.1 Persistent-object layout

Each protected file is represented by ordinary lower-filesystem state rather than an in-kernel extent map. The data sidecar stores encrypted records; the metadata sidecar stores logical-block to record mappings and authentication material; xattrs store compact metadata such as logical size and the authenticated envelope. This layout is intentionally inspectable for fault-injection experiments: a harness can copy or corrupt a backing directory, restore an older sidecar, or mutate an xattr without requiring a custom block device.

The layout also creates concrete assumptions. AEGIS-Q relies on the lower filesystem to honor `fdatasync` on the data and metadata sidecars and to preserve xattr updates according to its documented semantics. It does not claim that every directory-entry or rename edge case has been exercised. The paper therefore separates the implementation invariant—data is synced before a mapping is published—from the stronger certification claim that every possible lower-filesystem crash interleaving has been tested.

4.2 Concurrency and unsupported modes

The evaluated path uses ordinary FUSE read, write, flush, fsync, open, and release operations. Recent hot-path cleanup moves repeated policy parsing, same-daemon metadata reloads, sidecar EOF probes, read/write scratch allocation, and redundant read-side zeroing out of common mounted-path work. Strict mode still drains coalesced writes before sync returns; epoch mode can group selected publication barriers while replaying only committed prefixes. This supports the retained remount and SQLite selected-boundary experiments, not a general high-throughput filesystem claim.

The implementation is not presented as a full POSIX conformance layer. In particular, the paper does not claim validated shared `mmap` semantics, all possible concurrent-writer races, cross-process byte-range lock compatibility, or arbitrary rename/directory-fsync crash behavior. Those modes are common sources of filesystem bugs, and the review correctly identified them as missing evidence. Treating them as unsupported or unvalidated is preferable to silently inheriting a POSIX claim from FUSE.

4.3 Telemetry plumbing

The admission logic is implemented as ordinary C state rather than a separate daemon that invents scheduling results. It records input queue depth, batch size, slack, telemetry age, and route decision. The mounted-FUSE throttle path reads a telemetry file and applies hysteresis before sleeping inside the write-flush path. The tegrastats and CUPTI harnesses differ only in how they populate that telemetry input. This shared path is the reason the evaluation can claim same-run telemetry-to-FUSE wiring; it is still a storage-runtime result rather than an external application scheduler.

Table 5: Threat classes and mechanism boundaries.

Threat	Mechanism	Boundary
Backing-store read	AES-GCM ciphertext with per-file DEK	Assumes mount credential remains secret
Backing-store tamper	AEAD tags, HMAC envelope, HMAC checkpoint	Denial of service remains possible
Whole-directory replay	Optional external TPM anchor can fail closed after advance	File-only witness is replayable; full rollback resistance is not claimed
Crash during update	Ordered publication plus selected daemon/app cutpoint tests	Physical power-loss and drive-cache certification remain open
GPU side channel	No protection mechanism claimed	Integrity parity is not constant-time evidence
Privileged local attacker	Out of scope	Kernel/driver/FUSE process are trusted

5 Security and Recovery Scope

5.1 Threat model

AEGIS-Q considers an attacker who can read, copy, modify, and replay the backing directory but cannot obtain the mount credential or compromise the running kernel, FUSE daemon, CUDA driver, or cryptographic libraries. Under that scope, AES-GCM protects the confidentiality and authenticity of published block records, while the HMAC-protected key envelope and checkpoint detect unauthenticated metadata changes. The adversary can still deny service, delete records, replay a complete file-backed snapshot, or exploit vulnerabilities outside the FUSE process. We make no claim against a privileged local attacker that can read process memory, substitute the CUDA driver, alter the mount executable, or use GPU microarchitectural side channels.

The accepted encrypted state consists of a validated DEK envelope, a complete journal mapping, and ciphertext records referenced by that mapping. A tag failure causes a read to fail. A malformed or unauthenticated envelope/checkpoint causes open to fail. Table 5 states the threat classes these mechanisms address and the boundaries left outside the paper. These behaviors are necessary for a safe FUSE format, but they are not a formal proof of crash consistency. In particular, xattr atomicity and directory durability are delegated to the underlying filesystem.

5.2 Replay protection is conditional on an external anchor

The source tree contains two anchor modes. The file mode is only an integrity witness; copying the directory copies the witness. The corrected crash/replay run therefore reports four of four file-backed snapshot restores as *rollback visible*, not as successful protection. This negative result is important: an HMAC alone does not prevent rollback when its entire state is replayable.

The hardware mode stores an authenticated prefix record in an owner-authorized TPM NV index: administrators create and rotate the index outside the daemon, and AEGIS-Q

reads or writes only that record. PCR binding is a transient probe rather than a persistent filesystem policy; a software update or PCR change therefore requires explicit reprovisioning/resealing. Backup or migration cannot copy only the backing directory; it needs a fresh authorized anchor bootstrap. Lost mount credentials are unrecoverable. Missing index, authorization failure, TPM I/O failure, digest mismatch, or a TPM sequence ahead of local state fail closed; a TPM anchor behind local state is treated as a lagging witness and must be refreshed rather than as disk rollback. The evaluated claim is only replay-after-advance fail-closed behavior, including the SQLite same-store campaign in Section 6.3; it does not assert that an NV index is a signer, report invented TPM latency, or claim persistent PCR binding.

5.3 Side channels and application semantics

GPU SIMT execution, cache behavior, memory coalescing, occupancy, and contention can all leak information. Simulated timing and TVLA traces are excluded from this paper. The GPU integrity test checks output parity with OpenSSL; it says nothing about constant-time execution, physical power/EM leakage, or cross-tenant GPU channels.

Likewise, the clean-remount test validates one encrypted 128 KiB round trip through the final binary. It does not certify SQLite WAL, `mmap`, locking, crash recovery across every cut point, or the complete POSIX contract. The current security evidence is local to the edge runtime: FUSE self-tests, clean remount, managed-memory smoke, and GPU integrity parity show the implemented boundary, not statistical security, cloud/privacy economics, or remote key-management behavior.

Application semantics are separated from storage-record integrity. AEGIS-Q can reject a corrupted block tag, but it cannot infer whether an application-level transaction was acknowledged unless the experiment supplies an external oracle. The daemon power-fault matrix adds final-binary SIGKILL at selected one-block storage boundaries, while the SQLite rows supply application commit oracles. A complete database certification would need broader workloads, multiple cache modes, physical power loss, kernel crash, and drive-cache timing.

6 Evaluation

6.1 Methodology and claim scope

The evaluation follows the same spine as the design: RQ2 and RQ3 test the edge-runtime thesis, while RQ1, RQ4, and RQ5 bound the storage and recovery claims that make the placement result meaningful. It answers five research questions:

- **RQ1: Correctness of authenticated publication.** Does the implemented format preserve nonce/generation, publication, and metadata-auth invariants under remount, fault, and tamper campaigns?

- **RQ2: CPU/GPU/PQC placement.** When should a UMA edge runtime keep AES-GCM file encryption on the CPU, and when can ML-KEM/cuPQC key-plane maintenance use the GPU?
- **RQ3: Mounted application behavior.** Does storage-visible admission recover SQLite latency under secure-storage pressure, and do scoped append-log/cache-manifest workloads survive remount?
- **RQ4: Replay and recovery oracle.** Which daemon/app crash and TPM replay-after-advance outcomes are supported, and which rollback or power-loss claims remain outside scope?
- **RQ5: Cost boundaries and sensitivity.** What strict-publication cost appears against mode-aligned baselines, and which controller/key-plane/anchor mechanisms account for the result?

The design/evaluation contract is one closure per main mechanism: CPU/GPU/PQC placement through data-plane negative controls and key-plane measurements; QoS through SQLite recovery and kernel-control comparisons; managed-memory scope through executor-local diagnostics; format through generation/tamper checks; D/J/C publication through oracle-labeled cut points; and TPM replay checks through replay-after-advance.

Primitive-placement numbers come from the microbenchmark harness; the mounted rekey result comes from the key-plane workflow harness. Both refuse unsupported configurations rather than extrapolating missing rows.

The platform was an NVIDIA Jetson AGX Thor Developer Kit with 14 ARM CPU cores, a 1 TB WD PC SN5000S NVMe device, Linux 6.8.12-tegra, CUDA 13.0.48, liboqs 0.15.0, FUSE3 3.14.0, and cuPQC; it is not an Orin Nano result. Retained O4 tegrastats logs give same-run power/thermal observations: mean VIN is 29.1 W for SQLite QoS, 27.5 W for key-plane rekey, and 28.4/27.4 W for AEGIS-Q/dm-crypt frozen rows, with max $t_j=49.0C$. These are observations, not energy-efficiency claims.

Figure 5 is the evaluation spine: AEGIS-Q keeps AES-GCM data on the CPU, uses the GPU only for a mounted maintenance workflow with explicit slack, and recovers bounded SQLite pressure while giving up background bandwidth. The strict frozen-contract row is a cost boundary, not the headline claim. Table 6 states what each experiment can establish. The frozen filesystem contract fixes 4 KiB 70/30 random read/write, `psync`, per-write `fdatasync`, queue depth 1, one client, a 1 GiB file, fixed mounts/platform, and five repetitions. A same-profile attribution probe confirms that the strict path still pays data-sidecar, journal-sidecar, and marker/checkpoint durability boundaries per write-publication cycle. The matched dm-crypt row runs on the same warm-cache contract: median throughput 9.03 MiB/s and median p99/p99.9 latency 264/823 μs ; fsencrypt remains environment-blocked because `CONFIG_FS_ENCRYPTION` is unset and the root ext4 encrypt feature is unavailable.

X6/O3 answer strict-path practicality without upgrading crash claims. X6 changes marker/checkpoint publication

Table 6: Evaluation scope by research question.

RQ / exp.	Establishes	Does not establish
RQ1 FUSE	Format functions after clean remount	Full SQLite/crash certification
RQ1 hash	CUDA and OpenSSL digests agree	Side-channel resistance
RQ5 Frozen contract	Plain/gocryptfs/dm-crypt/AEGIS-Q warm; fsencrypt unavailable by env. proof	fsencrypt throughput and cold-cache rows remain unavailable
RQ2/RQ5 UMA	Managed allocation and host/device prefetch complete	Storage-DMA or O_DIRECT-to-UVM
RQ5 eBPF	<code>nvme_complete_rq</code> diagnostic histogram	Mounted eBPF/io_uring path
RQ3 mounted telemetry	Telemetry reaches the FUSE write-throttle path	Application scheduler or non-storage p99 controller
RQ3 stress	4-writer tail-latency probe under pressure	Broad foreground workload suite
RQ3/RQ4 mounted apps	Secure append-log remount plus cache-manifest object publish/remount	Broad workload suite or non-storage p99 recovery
RQ4 file anchor	File snapshot rollback remains visible	TPM-anchor replay protection

from filesystem-wide `syncfs` to marker-file `fsync` (1,070 retained A2 calls), leaving data and journal barriers. O3 keeps strict per-write publication as the D/J/C barrier boundary and admits hybrid epoch grouping only for concurrent or batched writes that can share a barrier. The grouped row cuts four-client syncs from eight to six and improves p99 from 47.6 to 40.1 ms and throughput from 9.05 to 9.72 MiB/s; the fault-matrix row cuts syncs from eight to five but worsens p99 to 53.1 ms, so this is conditional admission, not a general fast path. Single-client epoch still pays a data plus epoch-log barrier; kernel-assist remains future work.

6.2 CPU for bulk data, GPU for batched ML-KEM

The diagnostic placement harness explains the key asymmetry behind Figure 5(c). For a 16 MiB AES-256-GCM input, CPU/OpenSSL reaches 1.61 GB/s (observed 1.59–1.63 GB/s), whereas the managed-buffer GPU path reaches 0.172 GB/s (0.172–0.172 GB/s). The GPU result includes managed-buffer staging and stream synchronization. The retained 4 KiB–256 KiB data-plane negative-control rows amplify the same effect, with the current CUDA path 51–2061 $\times$ slower than CPU. This is why AEGIS-Q’s data plane is CPU-first; a GPU label alone is not a performance result.

For a batch of 4,096 ML-KEM-768 key generations, liboqs on CPU reaches 64.8 K operations/s (64.3–64.9 K), while cuPQC reaches 1.50 M operations/s (1.49–1.51 M), a 23.2 $\times$ ratio of medians. The comparison is a primitive microbenchmark on one platform, not a claim that a FUSE write becomes 23.2 $\times$ faster. It supports the narrower policy of reserving GPU work for sufficiently large, independent key-plane batches.

The mounted ML-KEM-768 key-plane workflow opens 1,024 encrypted files and refreshes their HMAC-protected envelopes. In the five-run methodology row, CPU-only refresh takes 24.575 ms (41.7 K files/s); with explicit slack,

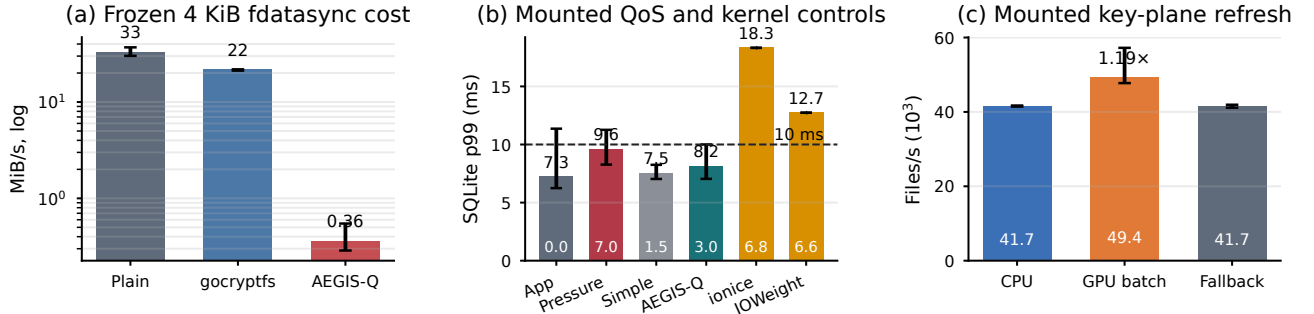


Figure 5: Evaluation summary generated from retained validation JSON. Panel (a) shows the strict frozen-contract cost boundary for RQ5; Panel (b) shows mounted SQLite QoS and kernel-control rows for RQ3; Panel (c) shows the mounted open-file key-plane refresh workflow for RQ2.

admission selects the GPU and finishes in 20.729 ms (49.4 K files/s), a $1.186\times$ median speedup; with zero slack and high memory-pressure telemetry, fallback remains CPU-equivalent at 24.572 ms. The X11 break-even model explains why the workflow gain is modest: primitive cuPQC is faster by batch 16, but mounted envelope/FUSE/admission overhead shifts the positive workflow break-even to roughly 760–812 refreshed files, and an added 5 ms of UMA or staging pressure shifts it to roughly 1.09–1.10K files. This supports only maintenance-visible open-file envelope refresh, not hardware-backed credential release, a persistent KEM hierarchy, or non-storage foreground recovery.

These measurements produce the main placement rule used by the design: AES-GCM data writes are latency-sensitive and remain CPU-first, while ML-KEM batches are elastic and may use the GPU only when producer slack and telemetry permit it. The result would be weaker if the paper reported only the faster GPU primitive; the important systems observation is the boundary between the data plane and the slack-tolerant maintenance plane.

6.3 Correctness and negative controls

The GPU integrity test passes leaf, Merkle, and empty-tree parity cases against OpenSSL. The FUSE self-test covers journal/checkpoint/anchor units and a generation-replay regression. The retained generation fault matrix covers partial update, clean remount, torn tails, stale snapshots, journal loss, older-generation append, and duplicate-generation replay across strict and epoch modes; every row has an oracle verdict without silent-corruption or unexpected-liveness outcomes. X10 adds `UINT64_MAX/EFBIG` near-wrap refusal, publish-ticket reservation, and same-file/disjoint 4-thread/4-process mounted writer stress, but not arbitrary application-level concurrency. Clean remount byte-compares plaintext after `fsync`, and envelope tamper returns `EKEYREJECTED`.

The POSIX-scope audit records a narrow mounted-workload interface: ordinary read/write/`fsync`, scoped rename, directory `fsync`, no-open hard links, symlinks, openfd truncate/fallocate, user xattrs, and bounded writer races

pass without mixed plaintext. Encrypted-file `mmap/msync`, open-subtree retargeting, arbitrary byte-range transactions, lower-filesystem xattr atomicity, and full crash-time POSIX visibility are rejected, scoped, or delegated.

Auxiliary storage/GPU diagnostics are deliberately scoped. The host-pinning and managed-storage probes show same-buffer CPU/GPU checksum agreement plus CUDA pointer diagnostics, but they do not prove UVM migration suppression or final FUSE data-path DMA.

The file-backed anchor produces the deliberately negative result: restoring an entire backing-directory snapshot is classified as `rollback_visible`, i.e., the previous committed baseline remains readable. The result remains a negative control rather than rollback resistance.

The TPM runs record anchor latency, PCR reads, NV public attributes for index `0x01500010`, and a transient PCR-policy probe. The hardware replay matrix labels stale disk/new TPM, new disk/stale TPM, missing index, changed PCRs, authorization failure, interrupted NV update, and normal replay-after-advance. This is replay-after-advance evidence, not a claim that the persistent filesystem anchor itself is PCR-sealed.

The broader crash/replay matrices are selected-boundary evidence, not complete app-level crash coverage. The daemon campaign kills the final binary at D/J/C, anchor, `fsync`, remount, and authenticated-read cut points; application rows cover SQLite and `dbm.dumb` TPM replay. X1 adds three loopback ext4/device-mapper lower-block `error` rows; after `e2fsck/remount`, pre-commit rows recover the previous committed payload and the post-`fsync` row recovers the latest committed payload. This is not physical power-loss, kernel-crash, drive-cache, arbitrary-workload, or multi-block raw-write certification.

The scoped mounted-application rows are not synthetic fio rows. The secure append-log macro appends ordered 16 KiB records in strict and epoch modes, remounts, and verifies hashes/order; retained p99 is 5.93 ms strict and 4.17 ms epoch. A cache-manifest workload publishes 24

hashed 16 KiB objects with closed-file rename plus directory `fsync`, remounts, and verifies all hashes, reporting 4.22/28.09 ms p50/p99 at 2.00 MiB/s. The multi-writer row shows the queue-depth limit: epoch grouping reduces traced syncs from 16 to 1 and replay reconstruction remains sub-ms, but p99.9 worsens to 67.90 ms versus 61.54 ms. These are scoped mounted patterns, not a broad workload suite or non-storage application QoS claim.

Recovery/replay outcomes support the mounted runtime boundary; they are not the paper’s central QoS/placement result.

6.4 QoS Recovery, Ablations, and Sensitivity

The tegrastats/CUPTI runs show platform pressure reaching the mounted daemon: they record 8 and 5 daemon-throttled flushes, respectively. This remains wiring evidence rather than application-level p99 recovery or fast-notification by-pass.

The app-level result uses SQLite transaction latency because it exercises the mounted file-encryption path, foreground tail latency, and elastic background pressure together. Table 7 reports five-run hero medians and three-run kernel QoS controls. AEGIS-Q reports 8.15 ms p99, zero median misses, and 3.02 MB/s background progress; unthrottled storage reports 9.62 ms and 6.98 MB/s. This is a Pareto point rather than a free throughput improvement: the simple controller reaches 7.54 ms p99 but only 1.50 MB/s background progress. Kernel controls preserve background throughput but not the tail here: `ionice` reports 16.22 ms p99 with six misses, and `IOWeight` reports 12.57 ms p99 with seven misses. This is bounded storage-visible control, not a uniqueness or non-storage QoS claim.

The sensitivity bundle varies budget, sampling interval, threshold, queue depth, and writer intensity. The default storage-pressure controller uses 20 ms telemetry sampling, 0.70/0.60 enter/exit thresholds, and a two-sample hold; admission defaults are 4 KiB minimum GPU batches, 1.73 ms minimum producer slack, 100 μ s deadline margin, 250 ms telemetry-age limit, and 0.80 queue threshold. Baseline p99 is 6.4 ms; 80 ms sampling and 128 KiB chunks remain at 7.0/7.2 ms, high threshold disables throttling (0 sleeps, 8.8 ms), and two writers is fragile (12.3 ms, one miss). No-throttle/no-slack fallbacks pass, and hysteresis records 9 transitions and 8 flips. This keeps the claim on mounted storage-visible control, not an external application scheduler.

The ablation manifest then attributes cost to the authenticated FUSE format, SQLite recovery to elastic-write throttling, key-plane speedup to slack-gated GPU maintenance, and replay refusal to the external anchor.

7 Discussion and Limitations

7.1 Interpretation of the placement result

The useful result is asymmetry, not aggregate “GPU speedup.” CPU/OpenSSL fits AES-GCM data writes, while

Table 7: SQLite p99 under mounted secure-storage pressure. Hero rows are five-run medians; kernel rows are 3 retained runs each.

Row	Control	p99	Miss	MB/s	Evidence
App only	none	7.25	0.00	0.00	5-run med.
Unthrottled	none	9.62	0.00	6.98	5-run med.
Simple ctrl.	harness sleep	7.54	0.00	1.50	5-run med.
AEGIS-Q	storage class	8.15	0.00	3.02	5-run med.
Kernel ionice	ionice -c 3	16.22	6.00	7.25	3-run proof
Kernel IOWeight	IOWeight=10	12.57	7.00	6.73	3-run proof

GPU/cuPQC helps slack-tolerant ML-KEM maintenance. Under UMA contention, secure file encryption needs admission and fallback rules that see both cryptographic role and foreground storage slack; AEGIS-Q trades peak background throughput for explicit publication, placement, QoS, and recovery boundaries.

7.2 Security limits

The per-file envelope is authenticated and block records are AEAD-protected, but the mount password remains the root credential. There is no hardware-backed credential release, audited rotation protocol, privileged-attacker defense, or side-channel evaluation. TPM probes cover fail-closed replay-after-advance, not persistent PCR-bound key release.

7.3 Evaluation limits and roadmap

The primitive measurements cannot establish steady-state throughput, energy efficiency, or portability. The frozen matrix covers plaintext, gocryptfs, dm-crypt, and AEGIS-Q; fscrypt is environment-blocked by kernel/filesystem proof and is not reported as a throughput row. The matrix exposes authenticated-publication cost; it is not a full fscrypt, fs-verity, dm-integrity, OP-TEE, SPDk, or GPUDirect Storage comparison or free storage path.

AEGIS-Q is not a general POSIX filesystem: read-/write/`fsync`, scoped rename, directory `fsync`, no-open hard links, relative symlinks, and bounded writer races are evaluated; shared `mmap`, full link lifecycle, open-subtree retargeting, arbitrary byte-range transactions, and application-level concurrency are not deployment claims.

The crash story retains daemon crashes, SQLite/dbm recovery, append-log/cache-manifest remount, syscall-exact app-crash, and loopback ext4/device-mapper accepted-state recovery. It still lacks physical power-loss, kernel-crash, drive-cache, broad workload-suite, non-NVIDIA UMA, and side-channel evidence.

Kernel assistance is future work: D/J/C barrier issuance could move to a helper, but only if a same-format kernel path preserves the recovery oracle; Merkle/integrity jobs must stay independent, batched, slack-tolerant, and format-compatible.

7.4 Deployment takeaway

The evaluated envelope is local: SQLite, elastic background writes, append-log/cache-manifest remounts, authenticated FUSE, kernel/FUSE-daemon trust, and Jetson/CUDA/TPM dependencies. The SQLite point is the same storage-pressure case summarized by Figure 1 and Table 7, not a separate deployment anecdote. The portable lesson is the policy split: CPU AES-GCM publication, slack-gated PQC maintenance, and executor-local managed memory.

8 Related Work

AEGIS-Q separates storage protection, GPU/storage placement, and PQC key-plane work. fscrypt and dm-crypt are mature kernel baselines [4, 5], and gocryptfs is the closest user-space encrypted-filesystem comparison point [15]; FUSE overhead is a cost boundary, not a free kernel substitute. fs-verity and dm-integrity validate integrity-only boundaries, while AEGIS-Q combines authenticated FUSE records, storage-visible policy, optional GPU maintenance, and external replay checks.

The comparison is asymmetric rather than deleted: plaintext/lowerfs, gocryptfs, dm-crypt, and AEGIS-Q are mode-aligned measured rows; fscrypt is unavailable with proof, not an implied speedup. These direct same-contract rows expose publication and QoS costs, not kernel-replacement maturity; work that requires a different kernel, GPU file API, driver path, or security boundary remains related work rather than an artificial throughput row.

GPU/storage systems validate different contracts. FlashNeuron and Fastensor stage large GPU workloads [1, 17]; SnuQS manages out-of-core capacity [12], while SnuQS alone is not filesystem-format evidence. Speculative GPU encryption, SPDK, GPUDirect Storage, and fscrypt-GPU optimize masks, movement, GPU access, or throughput, but not AEGIS-Q's FUSE records, xattrs, external anchors, D/J/C publication, TPM replay, or storage-visible QoS contract [1, 3, 17, 18]. GPUstore, GPUfs, GeminiFS, and GPU4FS expose storage services or filesystem acceleration to GPU-side execution [6, 13, 14, 16]; GPUDirect Storage exposes a direct storage/GPU data path through cuFile [9]. AEGIS-Q keeps sidecar I/O and asks when a mounted encryption runtime should admit optional GPU key-plane work without changing the authenticated format. OP-TEE/dm-crypt use different boundaries [4, 11]; CUDA/cuPQC evidence is platform-specific.

Linux storage QoS mechanisms such as blk-throttle, io.latency/I0.cost, BFQ, and CFQ operate below file-encryption semantics; AEGIS-Q's *ionice/I0Weight* rows are controls, not an exhaustive scheduler study. F2FS, ScaleFS, FastCommit, encrypted databases, integrity trees, and secure logs place append/replay oracles at other boundaries; AEGIS-Q's analogue is a scoped FUSE sidecar epoch log and bounded leader/follower barrier.

ML-KEM establishes shared secrets rather than per-block bulk encryption [8]; post-quantum encrypted-file-system

cost models reinforce that primitive choice is not a mounted runtime [2]. AEGIS-Q keeps AES-GCM in the data plane and uses ML-KEM only for batched key-plane work. OP-TEE/TrustZone isolates attested code paths [11]; AEGIS-Q does not provide that TEE boundary, and stronger TPM rollback claims still need explicit NV authorization, key-release policy, and recovery semantics. It is not a replacement for kernel encryption, POSIX filesystems, TEEs, or GPU-storage stacks, but a secure edge-storage runtime.

9 Conclusion

AEGIS-Q treats secure edge file encryption as a placement, QoS, and recovery-boundary runtime under secure-storage pressure, not blanket GPU offload. Its format keeps AES-GCM writes on CPU, admits ML-KEM/cuPQC maintenance only with slack, and reports 8.15 ms SQLite p99 with 3.02 MB/s background progress on Thor. Accelerator throughput matters only after publication order, QoS, and replay boundaries are explicit.

References

- [1] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W. Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 387–401.
- [2] Daniel J. Bernstein, Tung Chou, Tanja Lange, and Christine van Vredendaal. 2024. Predicting Performance for Post-Quantum Encrypted-File Systems.
- [3] Vandeir Eduardo, Luis C. Erpen de Bona, and Wagner M. Nunan Zola. 2019. Speculative Encryption on GPU Applied to Cryptographic File Systems. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*. 93–105.
- [4] Clemens Fruhwirth. 2005. New methods in hard disk encryption: dm-crypt and LUKS. *Linux Journal* 2005, 138 (2005).
- [5] Michael Halcrow, Theodore Ts'o, et al. 2015. Ext4 Encryption: Design and Implementation of Linux Filesystem Encryption. *Proceedings of the Linux Symposium* (2015).
- [6] Julian Maucher, Steven Diethardt, Lukas Steffen, Alexander Girol, Harald Obermaier, and Andreas Koch. 2024. GPU4FS: Full-Scale File System Acceleration on GPU. In *Architecture of Computing Systems (ARCS 24)*.
- [7] National Institute of Standards and Technology. 2015. *FIPS 180-4: Secure Hash Standard (SHS)*. Technical Report. NIST.
- [8] National Institute of Standards and Technology. 2024. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Technical Report. NIST.
- [9] NVIDIA Corporation. 2026. *NVIDIA GPUDirect Storage: cuFile API Reference Guide*.
- [10] NVIDIA Corporation. 2026. *NVIDIA Jetson Linux Developer Guide: Platform Power and Performance*.
- [11] OP-TEE. 2018. OP-TEE: Open Portable Trusted Execution Environment Specification. In *GlobalPlatform Technology Conference*.
- [12] Daeyoung Park, Heehoon Kim, Jinpyo Kim, Taehyun Kim, and Jaejin Lee. 2022. SnuQS: Out-of-Core Cache Staging for Virtualized Memory Storage Systems. In *Proceedings of the International Conference on Supercomputing*.
- [13] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. 2025. GeminiFS: A Companion File System for GPUs. In *29th USENIX Conference on File and Storage Technologies (FAST 25)*. 221–236.
- [14] Mark Silberstein, Bryan Ford, Idit Keidar, and Emmett Witchel. 2013. GPUfs: Integrating File Systems with GPUs. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 13)*. 485–498.
- [15] Jakob Spennberg et al. 2017. gocryptfs: A Modern FUSE-based Encrypted Filesystem. In *USENIX Security Association*.
- [16] Weibin Sun, Robert Ricci, and Matthew L. Curry. 2012. GPUstore: Harnessing GPU Computing for Storage Systems in the OS Kernel. In *Proceedings of the 5th Annual International Systems and Storage Conference (SYSTOR 12)*.
- [17] Jia Wei, Xingjun Zhang, Longxiang Wang, and Zheng Wei. 2023. Fastensor: Optimize the Tensor I/O Path from SSD to GPU for Deep Learning Training. *ACM Trans. Archit. Code Optim.* 20, 4 (2023).
- [18] Xingjun Zhang, Longxiang Wang, et al. 2023. fscrypt-GPU: GPU-Accelerated Filesystem Encryption for High-Throughput Secure Storage. In *ACM Symposium on Cloud Computing*.